Hash tables solutions

11-E01 Load factor and bucket events

Derivation

$\alpha = 18/12 = 1.5$, giving expected cost $\Theta(1 + \alpha)$. All keys enter a specified bucket with probability 12^{-18}. Sharing any bucket is a union of twelve disjoint events, with probability $12 \cdot 12^{-18} = 12^{-17}$. Multiplying these exact probabilities requires independent key-to-bucket choices.

Result and check

An unsuccessful search checks 1.5 records in expectation, plus fixed hashing/indexing overhead. Theta notation does not specify an exact time of 2.5. A worst-case operation can still scan 18 records. Simple uniform hashing assumes independent uniform choices; concentrated actual key assignments do not automatically satisfy this analysis.

11-E02 Modulo collisions at m=2^p

Derivation

0 and 8 both have remainder 0 modulo 8; 6 and 14 both have remainder 6. In binary, k mod 2^p keeps only the lowest p bits, so higher-bit differences do not affect the bucket.

Result and check

Any equal low three bits cause a collision. Strong low-bit patterns can concentrate a power-of-two division hash in a few buckets.

11-E03 Deletion in open addressing

Derivation

Search for 8 checks slot 1 first. If it sees NIL/EMPTY and stops, it incorrectly reports absence. Erasing 1 has broken 8's probe path.

Result and check

Mark slot 1 DELETED and continue to slot 2, or reinsert the subsequent cluster when deleting. A correct method must let search cross the gap.

Hash tables solutions (continued)

11-E04 Exact finite universal-family probability

Derivation

The 20 choices (a,b) correspond bijectively to the 20 ordered distinct pairs (r,s), with r=b and s=a+b mod 5. Modulo 3 classes are {0,3}, {1,4}, and {2}. The numbers of ordered distinct pairs within them are 2+2+0=4.

Result and check

The exact collision probability is 4/20=1/5≤1/3. Universality requires at most 1/m, not equality, uniform output marginals, or independent collision events. Fix x,y before choosing (a,b).

11-E05 Double-hashing coverage period

Derivation

For step 8, gcd(8,12)=4 and the period is 12/4=3. Probes are 1,9,5,1, reaching only three slots. Step 5 is coprime to 12 and has period 12, giving 1,6,11,4,9,2,7,0,5,10,3,8 before returning to 1.

Result and check

Even if 1,9,5 are full, step 8 cannot establish that the whole table is full. Its probe specification fails to cover the other slots. Returning home means 12 divides i・step; canceling the gcd gives least positive i=12/gcd and the exact period.

11-E06 Perfect hashing still compares original keys

Derivation

Buckets are 1={31}, 2={20,26}, 3={9,15}, 4={4}, with 0 and 5 empty. Second-level sizes total 1+4+4+1=10. Bucket 2 has 20 at slot 0 and 26 at slot 2; bucket 3 has 9 at slot 1 and 15 at slot 3. Both queries 26 and 14 go to first-level 2, second-level 2, and read 26.

Result and check

26 matches and succeeds; 14 differs and must be absent. Perfect hashing guarantees distinct positions for keys inside K, not for all queries outside K. Comparing the actual key preserves dictionary correctness. Empty buckets or EMPTY slots also reject directly.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Hash tables solutions (continued)

Verification code appendix: hash_checks.py

```
"""Finite probability enumeration and dictionary behavior with genuine collision handling."""
from itertools import combinations, permutations, product
from fractions import Fraction
from math import gcd, log
import random

EMPTY=object()
DELETED=object()


def affine(key,a,b,p,m):
    return ((a*key+b)%p)%m


class OpenTable:
    def __init__(self,m):
        self.slots=[EMPTY]*m
    def search(self,key):
        m=len(self.slots)
        for i in range(m):
            j=(key+i)%m
            record=self.slots[j]
            if record is EMPTY:
                return None,i+1
            if record is not DELETED and record[0]==key:
                return j,i+1
        return None,m
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash tables solutions (continued)

```
    def insert(self,key,value):
        first=None
        m=len(self.slots)
        for i in range(m):
            j=(key+i)%m
            record=self.slots[j]
            if record is not EMPTY and record is not DELETED and record[0]==key:
                self.slots[j]=(key,value)
                return j,i+1
            if record is DELETED and first is None:
                first=j
            if record is EMPTY:
                target=first if first is not None else j
                self.slots[target]=(key,value)
                return target,i+1
        if first is not None:
            self.slots[first]=(key,value)
            return first,m
        raise OverflowError
    def delete(self,key):
        index,_=self.search(key)
        if index is not None:
            self.slots[index]=DELETED
    def contents(self):
        records=[r for r in self.slots if r is not EMPTY and r is not DELETED]
        assert len(records)==len({r[0] for r in records})
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash tables solutions (continued)

```
        return dict(records)


def perfect_build(keys,p,rng):
    keys=list(keys)
    n=len(keys)
    if not n:
        return p,0,0,[]
    while True:
        a,b=rng.randrange(1,p),rng.randrange(p)
        buckets=[[] for _ in range(n)]
        for key in keys:
            buckets[affine(key,a,b,p,n)].append(key)
        if sum(len(bucket)**2 for bucket in buckets)<4*n:
            break
    secondary=[]
    for bucket in buckets:
        size=len(bucket)**2
        if not size:
            secondary.append(None)
            continue
        while True:
            aa,bb=rng.randrange(1,p),rng.randrange(p)
            table=[None]*size
            collision=False
            for key in bucket:
                index=affine(key,aa,bb,p,size)
                if table[index] is not None:
                    collision=True
                    break
                table[index]=key
            if not collision:
                secondary.append((aa,bb,table))
                break
    return p,a,b,secondary


def perfect_contains(built,key):
    p,a,b,secondary=built
    if not secondary:
        return False
    bucket=secondary[affine(key,a,b,p,len(secondary))]
    if bucket is None:
        return False
    aa,bb,slots=bucket
    return slots[affine(key,aa,bb,p,len(slots))]==key


def main():
    families=0
    for p in [5,7,11]:
        for m in range(1,p+3):
```

Hash tables solutions (continued)

```
        family=[(a,b) for a in range(1,p) for b in range(p)]
        for x,y in combinations(range(p),2):
            count=sum(affine(x,a,b,p,m)==affine(y,a,b,p,m) for a,b in family)
            assert Fraction(count,len(family))<=Fraction(1,m)
        families+=1
assert sum(affine(0,a,b,5,3)==affine(1,a,b,5,3) for a in range(1,5) for b in range(5))==4
assert 13*159==2067 and (2067%256)>>4==1
for m in range(2,65):
    for step in range(1,m):
        assert len({i*step%m for i in range(m)})==m//gcd(m,step)
for w in range(1,9):
    m=2**w
    assert len({i*(i+1)//2%m for i in range(m)})==m
assert [i*i%7 for i in range(7)]==[0,1,4,2,2,4,1]
table=OpenTable(7)
for key in [1,8,15]:
    table.insert(key,str(key))
table.delete(1)
assert table.search(15)==(3,3)
assert table.insert(8,'updated')==(2,2)
assert table.insert(22,'new')==(1,4)
assert table.contents()=={22:'new',8:'updated',15:'15'}
full=OpenTable(7)
for key in range(7):
    full.insert(key,key)
for key in range(7):
    full.delete(key)
assert full.search(101)==(None,7)
assert full.insert(3,'reused')==(3,7)
rng=random.Random(20261024)
table=OpenTable(7)
reference={}
for step in range(5000):
    key=rng.randrange(21)
    operation=rng.randrange(3)
    if operation==0:
        before=table.slots[:]
        if len(reference)==7 and key not in reference:
            try:
                table.insert(key,step)
                raise AssertionError('full table must reject')
            except OverflowError:
                assert before==table.slots
        else:
            table.insert(key,step)
            reference[key]=step
    elif operation==1:
        table.delete(key)
        reference.pop(key,None)
    else:
        index,_=table.search(key)
        assert (index is not None)==(key in reference)
    assert table.contents()==reference
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Hash tables solutions (continued)

```
        for query in range(21):
            index,_=table.search(query)
            assert (index is not None)==(query in reference)
    exact_probe_permutations=0
    for m in range(1,8):
        orders=list(permutations(range(m)))
        for n in range(m):
            total=0
            for order in orders:
                for index,slot in enumerate(order,1):
                    if slot>=n:
                        total+=index
                        break
                exact_probe_permutations+=1
            assert Fraction(total,len(orders))==Fraction(m+1,m-n+1)
            assert Fraction(total,len(orders))<=Fraction(m,m-n)
    m,n=4,3
    permutations4=list(permutations(range(m)))
    success_total=0
    combinations4=0
    for probes in product(permutations4,repeat=n):
        occupied=set()
        for order in probes:
            for steps,slot in enumerate(order,1):
                if slot not in occupied:
                    occupied.add(slot)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash tables solutions (continued)

```
                    success_total+=steps
                    break
        combinations4+=1
    success_mean=Fraction(success_total,n*combinations4)
    assert success_mean==Fraction(47,36)
    assert float(success_mean)<=log(1/(1-n/m))/(n/m)
    sets=0
    for size in range(9):
        for keys in combinations(range(8),size):
            built=perfect_build(keys,11,rng)
            assert sum(len(entry[2]) for entry in built[3] if entry is not None)<4*size if
size else built[3]==[]
            for key in range(11):
                assert perfect_contains(built,key)==(key in keys)
            sets+=1
    sample=[4,9,15,20,26,31]
    buckets=[[key for key in sample if key%6==j] for j in range(6)]
    assert buckets==[[],[31],[20,26],[9,15],[4],[]]
    assert sum(len(group)**2 for group in buckets)==10
    assert (26%6,26%4)==(14%6,14%4)==(2,2) and 26!=14
    print(f"PASS affine_families={families} open_table_mixed_operations=5000
tombstone_duplicate_and_all_deleted=True exact_failed_probe_permutations=
{exact_probe_permutations} successful_mean={success_mean} perfect_keysets={sets}
absence_key_check=True")


if __name__=='__main__':
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study